

Tema 9 - Organización de la Memoria

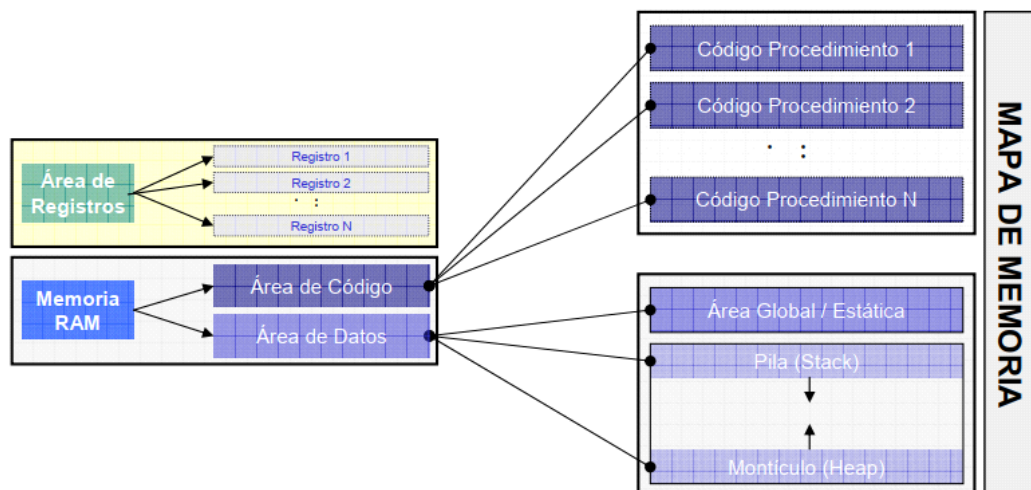
Profesor Ramón López-Cózar Delgado, Curso 21-22

Introducción a la gestión de memoria:

- El código intermedio aporta lo siguiente:
 - **Expresiones:** Descompone expresiones complejas en expresiones simples (unarias o binarias) introduciendo símbolos temporales.
 - **Tabla de Símbolos:** Están los símbolos que usa el programa fuente más los temporales introducidos en la fase de generación de código intermedio.
- En la generación de código es necesario establecer estrategias de organización para:
 - Asignar memoria a los símbolos (enlace).
 - Acceder a los valores almacenados (estado).
- Para el diseño del **mapa de memoria** hay que determinar las cuestiones siguientes:
 - Permitir o no la **recursividad**.
 - Qué hacer con los **valores locales** al salir de un procedimiento.
 - Uso o no de **valores no locales** en los procedimientos.
 - Técnica de **paso de parámetros** a los procedimientos.
 - Pasar o no **procedimientos** como **parámetros**.
 - Devolver o no **procedimientos** como **resultados**.
 - **Asignación** o no **dinámica de memoria** bajo el control del programa.
- Semántica de los procedimientos:
 - **Activación:** Ejecución del conjunto de instrucciones del procedimiento.
 - **Desactivación:** Salto a la instrucción siguiente inmediata que lo activó.
 - **Árbol de Activación:**
 - Nodo raíz: Programa Principal.
 - Nodos hijos: Procedimientos activados por el nodo ancestro o padre.
 - Pila de activación: Simula el control de activación, cuando aparece un subprograma introducimos en la pila la activación del mismo y cuando finaliza se extrae la activación del tope de la pila. En cada momento, la pila representa los procedimientos que han sido activados.

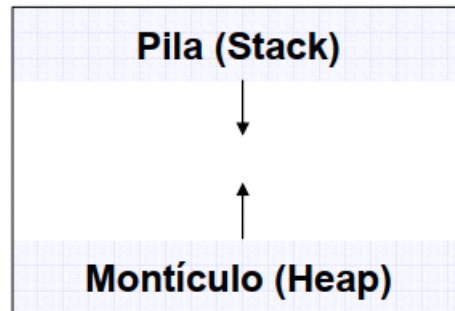
Organización de la memoria durante la ejecución:

- La memoria de un ordenador está dividida en:



- Esta distribución obedece a lo siguiente:
 - El punto de entrada de cada procedimiento se conoce en tiempo de compilación.

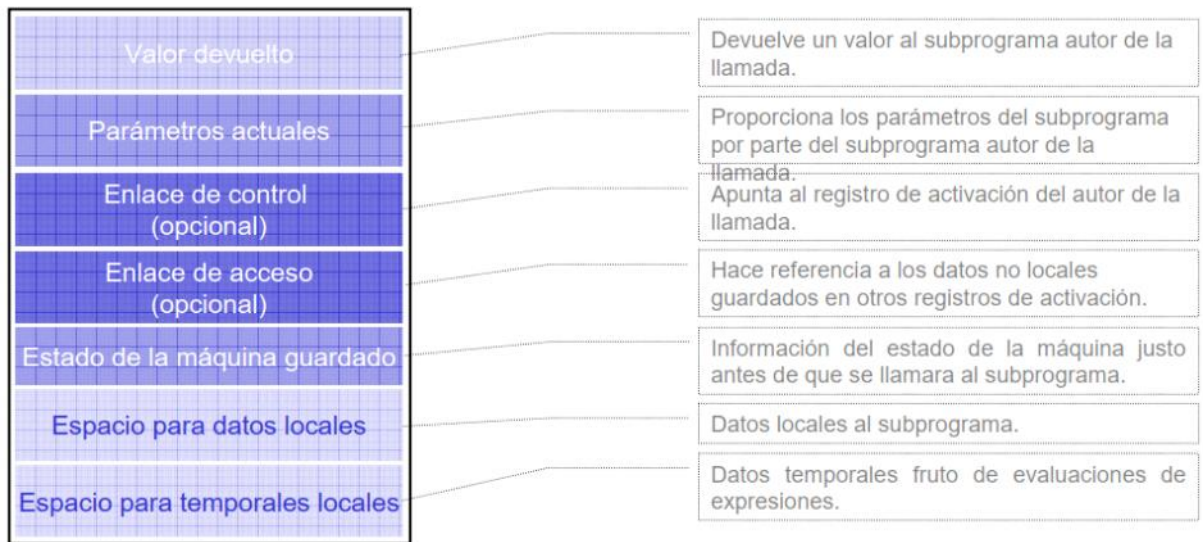
- Ciertos datos pueden asignarse en ubicaciones fijas de memoria antes de la ejecución.
 - En Fortran77 todos los datos son de esta forma, es decir, estáticos.
 - En Pascal serían las variables globales.
 - En C serían las variables estáticas y externas.
- El área de datos global de los procedimientos también es conocido en tiempo de compilación.
- El resto de información que depende de la ejecución del programa deberá almacenarse en otra zona denominada dinámica.
- Una organización típica del área de memoria para la asignación dinámica divide la memoria en dos áreas:



- **Pila (Stack):** Datos cuya asignación se corresponde con la gestión típica de pila de arquitecturas típicas en cuanto a llamadas y retornos de procedimientos.
- **Montículo (Heap):** Área reservada para asignación y liberación de datos cuyo control se realiza desde el propio programa.
- Un elemento importante de la asignación de memoria es el **registro de activación**, que es un bloque contiguo de información necesaria para cada ejecución de un procedimiento.
- Un **registro de activación** debe contener, como mínimo, la siguiente información:



- Detalles específicos del **registro de activación** dependerán de la arquitectura de la máquina objeto, de las propiedades del lenguaje que se esté compilando y de las preferencias establecidas en la construcción del propio traductor.
- Una composición general de un registro de activación sería...



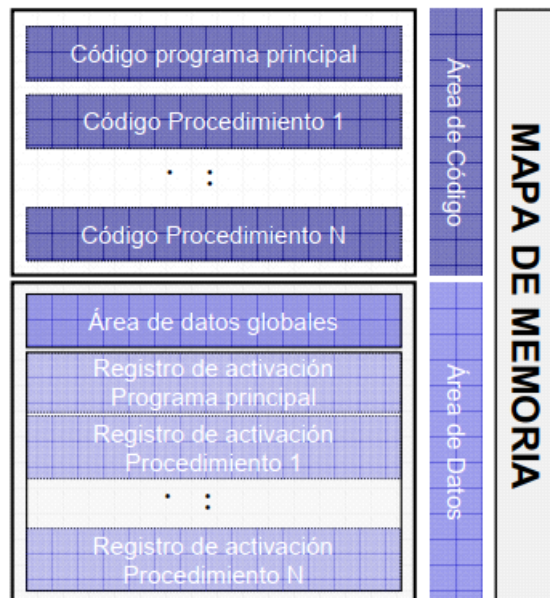
- El tamaño de cada uno de los campos del registro de activación es conocido cuando es activado, en general, el tamaño es conocido en tiempo de compilación, salvo los datos tipo array con dimensión en los parámetros actuales.
- Dependiendo del tipo de lenguaje, los registros de activación pueden ser asignados en el área estática, como en Fortran77, en el área de la pila, como en C y Pascal, o en el montículo, como es el caso de Lisp.
- Los registros del procesador también forman parte de la estructura de la memoria, si una arquitectura posee multitud de registros (como los procesadores RISC recientes), toda el área estática y los registros de activación completos pueden ser conservados en registros.
- Los procesadores poseen registros de propósito específico: contador de programa, apuntador de pila, apuntador de registro de activación, apuntador de argumento, etc.

Estrategias para la Asignación de Memoria

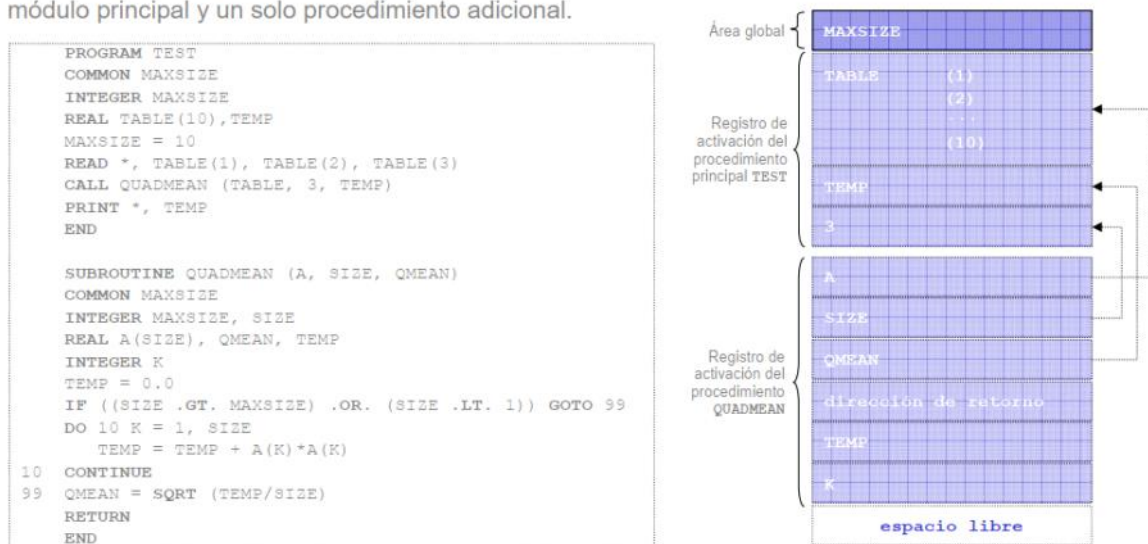
- **Asignación Estática:** Dispone la memoria para todos los objetos de datos en tiempo de compilación.
- **Asignación por medio de pila:** Trata la memoria en ejecución como una pila (stack).
- **Asignación por medio de montículo:** Asigna y libera la memoria conforme se necesita durante la ejecución a partir de un área de datos conocida como montículo (heap).

Organización Estática:

- Se caracteriza por que...
 - Los nombres se establecen en memoria en tiempo de compilación.
 - No es necesario un paquete de apoyo para la ejecución.
 - Todos los datos son estáticos y permanecen fijos en la memoria mientras dure la ejecución del programa
- Tiene las limitaciones...
 - El tamaño de los objetos de datos y las posiciones de memoria deben de conocerse en tiempo de compilación.
 - No se permiten llamadas recursivas.
 - No permite la creación dinámica de datos al carecer mecanismos para asignación y liberación de memoria durante la ejecución.
- Hay relativamente poco gasto general en términos de administración de información, la memoria está distribuida en dos áreas:
 - **Área de código:** Código del programa principal y de cada uno de los procedimientos de forma consecutiva.
 - **Área de datos:** Datos globales y registros de activación para cada uno de los procedimientos de forma consecutiva.



Ejemplo 9.1: Mapa de memoria del área de datos de un programa en Fortran77 constituido por un módulo principal y un solo procedimiento adicional.



Organización basada en pila (Stack):

- En los lenguajes que se permiten las llamadas recursivas, los registros de activación no pueden estar asignados de manera estática.
- Se tiene la pila de registros de activación, la gestión de llamadas se realiza mediante la gestión de una pila, también se la conoce como pila de ejecución o pila de llamadas.
- Además de almacenar los registros de activación es necesario contemplar la secuencia de llamadas a los procedimientos.
- En estas condiciones, según sea el ámbito de los procedimientos tendremos diferentes gestiones:
 - Gestión de pila sin procedimientos locales (Procedimientos globales)
 - Gestión de pila con procedimientos locales (Procedimientos anidados)

Pila sin Procedimientos Anidados:

- Gestión adecuada en lenguajes donde los procedimientos sean de ámbito global, como en el lenguaje C.
- La gestión de memoria mediante pila requiere dos cosas:
 - Apuntador hacia el registro de activación actual para permitir acceso a las variables locales.
 - Registro de la posición o tamaño del registro de activación inmediatamente precedente (el registro de activación del que lo llamó)

- El usuario introduce los valores 15 y 10

```
#include <stdio.h>

int x, y ;

int mcd ( int u, int v )
{
    if (v == 0)
        return u ;
    else
        return mcd (v, u % v) ;
}

int main()
{
    scanf ("%d%d", &x, &y) ;
    printf ("%d\n", mcd(x,y)) ;
    return 0 ;
}
```

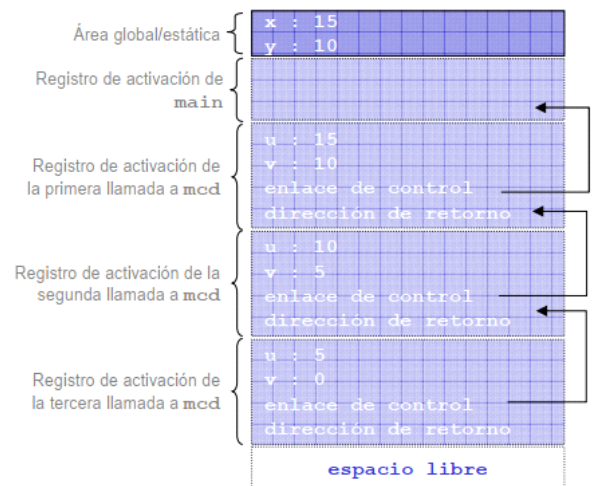


```
#include <stdio.h>

int x, y ;

int mcd ( int u, int v )
{
    if (v == 0)
        return u ;
    else
        return mcd (v, u % v) ;
}

int main()
{
    scanf ("%d%d", &x, &y) ;
    printf ("%d\n", mcd(x,y)) ;
    return 0 ;
}
```



- Cuestiones relacionadas con esta forma de organizar la memoria:
 - **Árbol de activación:** Representación jerárquica de las llamadas a procedimientos donde cada registro de activación se convierte en un nodo en este árbol.
 - **Acceso a nombres:** En una organización mediante pila, el acceso a variables locales y parámetros no se realizan mediante direcciones fijas sino a partir de desplazamientos desde el apuntador del marco actual.
 - **Secuencia de llamada:** Asigna un registro de activación e introduce información en sus campos.
 - **Secuencia de retorno:** Restablece el estado de la máquina para el procedimiento que llama y, de ese modo, continuar su ejecución.
- No hay una frontera clara entre quién hace las tareas correspondientes a la secuencia de llamada.
- Un reparto válido podría ser el siguiente:
 - **Procedimiento llamador**
 - Evalúa los parámetros reales.
 - Almacena la dirección de retorno y el valor anterior del tope de la pila en el registro de activación del subprograma llamado.
 - **Procedimiento llamado**
 - Guarda información de los registros y del estado de la máquina.
 - Asigna valores iniciales a sus datos locales y comienza la ejecución.
 - Coloca un valor devuelto a continuación del registro de activación del que lo llama.

- Restablece el tope de la pila y salta a la dirección de retorno mediante la información del campo de estado.
- El llamador puede copiar el valor devuelto por el llamado en su propio registro de activación.

Pila con Procedimientos Anidados:

- Gestión adecuada en lenguajes en que se permite anidamiento de procedimientos, como en Pascal.
- La técnica anterior no contempla referencias no locales y no globales.

Solución en el ámbito dinámico

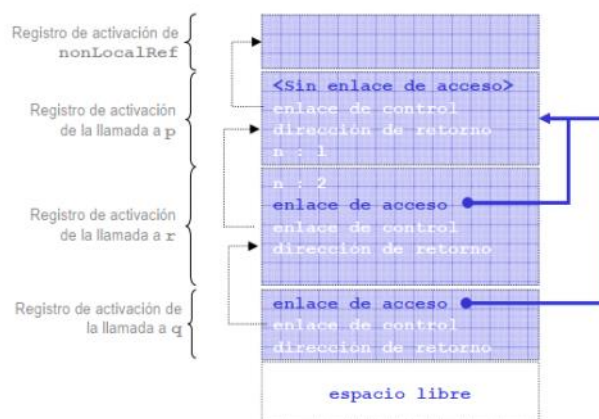
- Conservar las tablas de símbolos locales de cada procedimiento durante la ejecución para buscar un determinado identificador y, en su caso, calcular su desplazamiento.
 - **Problema:** Se soluciona de forma dinámica algo que puede conocerse en tiempo de compilación.

Solución en el ámbito estático

- Agregar un elemento de información extra de administración que se conoce como enlace de acceso o vínculo de acceso.

Enlace de Acceso

- Es una forma directa de implementar el ámbito cuando hay anidamiento de procedimientos.
- A cada registro de activación de los procedimientos se le añade un campo para el enlace de acceso que apunta al procedimiento que lo abarca.
- Para acceder a un nombre no local cuando se usa el enlace de acceso es necesario conocer la profundidad del anidamiento.



```

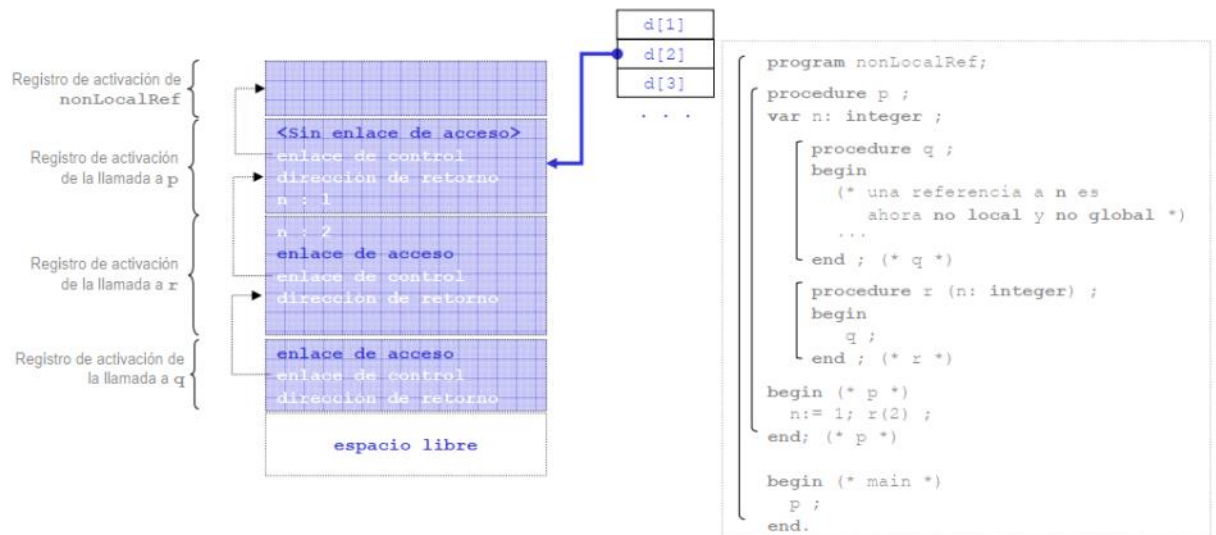
program nonLocalRef;
procedure p ;
var n: integer ;
begin
  procedure q ;
  begin
    (* una referencia a n es
       ahora no local y no global *)
    ...
  end ; (* q *)
  procedure r (n: integer) ;
  begin
    q ;
  end ; (* r *)
begin (* p *)
  n:= 1; r(2) ;
end; (* p *)

begin (* main *)
  p ;
end.

```

Estructura de datos tipo display

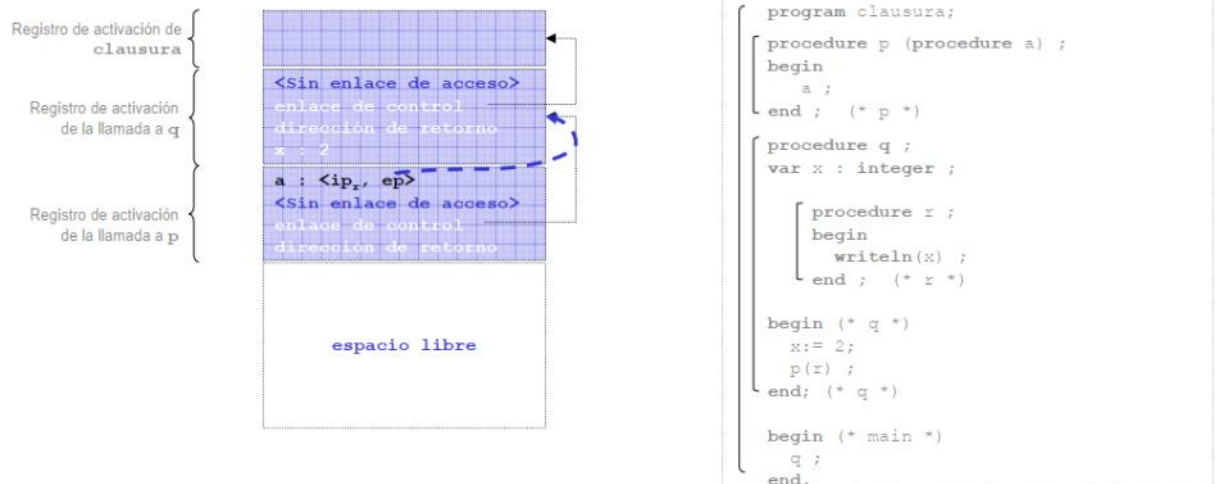
- Es una alternativa más rápida para acceder a los nombres no locales que el ofrecido por los enlaces de acceso.
- Consiste en utilizar una matriz d de apuntadores a registros de activación, un nombre no local A con profundidad de anidamiento i aparece en el registro de activación apuntado por el elemento $d[i]$ del display.
- Una solución sencilla para mantener actualizado el display es utilizar los enlaces de acceso además del display:
 - Como parte de las secuencias de llamadas y de retorno, el display se actualiza siguiendo la cadena de enlaces de acceso.
 - Cuando se sigue el enlace de acceso a un registro de activación a profundidad de anidamiento n , el display $d[n]$ se actualiza para que apunte a dicho registro de activación.
- En realidad, el display duplica la información de la cadena de enlaces de acceso.



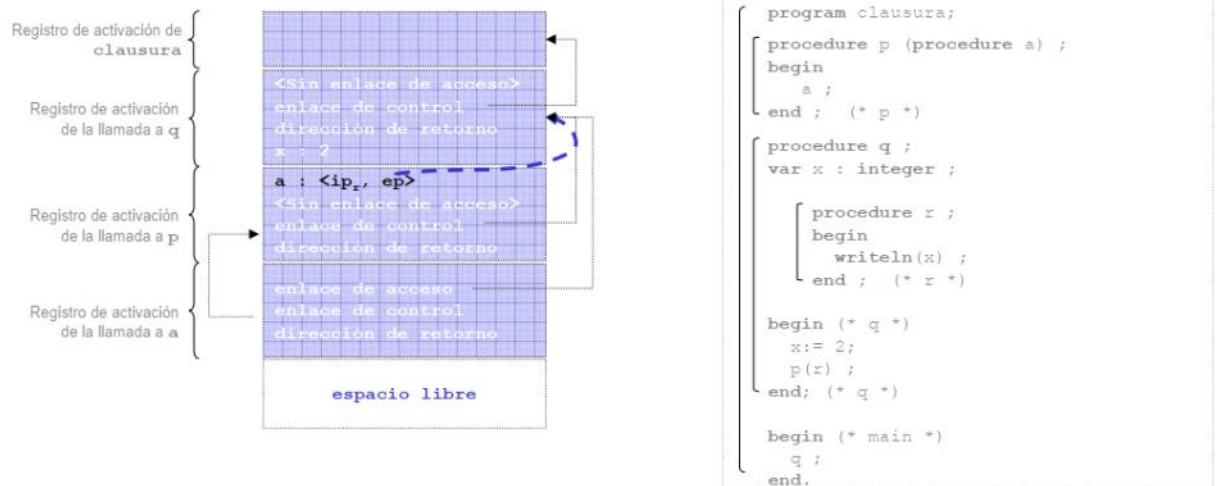
Pila con procedimientos como parámetros

- Algunos lenguajes permiten el paso de procedimientos como parámetros
 - Cuando se llama a un procedimiento que ha sido pasado como un parámetro, es imposible para un compilador generar código que permita calcular el enlace de acceso en el momento de la llamada.
 - Un procedimiento como parámetro es, inicialmente, un apuntador al código de dicho procedimiento, se denota ip_a al puntero de la instrucción del procedimiento a .
- Solución**
 - Incluir además del apuntador al código del procedimiento, un apuntador de ambiente (enlace de acceso) para resolver las referencias no locales, se denota ep al apuntador de ambiente.

Ejemplo 9.7: Mapa de memoria de un programa en Pascal con procedimientos anidados y un procedimiento como parámetro (justo después de la llamada al procedimiento p).



Ejemplo 9.7 (cont.): Mapa de memoria de un programa en **Pascal** con procedimientos anidados y un procedimiento como parámetro (justo después de la llamada al procedimiento **a**).



- A la hora de confeccionar un compilador, por simplicidad o uniformidad, es posible tratar los procedimientos ordinarios y los procedimientos dados como parámetros de manera equivalente mediante esta estrategia.
- Algunos lenguajes evitan este tipo de complicaciones:
 - C no posee procedimientos locales, aunque tiene variables y procedimientos como parámetros.
 - Modula-2 tiene una regla especial que restringe los procedimientos como parámetros a procedimientos que sean globales.
 - Ada no tiene procedimientos como parámetros.

▶ El resto del tema no fue impartido en este curso.